# CineComic-MML

## Phase 2: Speech Recognition (Whisper) - complete code

New and changed files for Phase 2, in the guide's folder structure. Add them on top of the Phase 1 project. **38 unit tests pass** (15 from Phase 1 plus 23 new ones). The tests use a fake backend, so the real Whisper model itself has **not** been run yet: do that on your machine (the first run downloads the model).

## What Phase 2 does

• Reads `data/metadata/{id}.json` and the 16 kHz WAV from Phase 1.

• Transcribes with Whisper (faster-whisper by default, openai-whisper optional) with word timestamps and optional VAD.

• Drops hallucinations: no-speech with low confidence, and known phrases such as 'thanks for watching' (kept in a *dropped* list for the report).

• Re-cuts segments to balloon size: split at sentence ends and long pauses, then at commas or the biggest pause so each line is at most 12 words and 4 s; merge fragments under 0.5 s.

• Sorts, removes overlaps, clamps to the audio length, then assigns segment_id 1, 2, 3 ... (the primary key for every later phase).

• Writes `data/subtitles/{id}.json` and `.srt`, evaluates and applies the gate. A video with no audio gets an empty transcript.

## Files in this document

| Status | File | Purpose |
|---|---|---|
| NEW | `configs/phase2.yaml` | Phase 1 settings plus the new asr and asr_gates sections (all thresholds live here). |
| NEW | `src/audio/postprocess.py` | Pure functions: filter hallucinations, split to balloon size, merge short fragments, fix the timeline. |
| NEW | `src/audio/srt_io.py` | Read and write SRT files with the srt library. |
| NEW | `src/audio/whisper_asr.py` | Whisper backends (faster-whisper, openai-whisper), run_asr() and the AsrStage. |
| NEW | `src/evaluation/asr_metrics.py` | WER, CER, start-time MAE, hallucination rate, ground-truth evaluation. |
| NEW | `src/evaluation/asr_model_study.py` | tiny/base/small/medium study: WER vs real-time factor, writes a CSV. |
| NEW | `tests/test_phase2.py` | 23 new unit tests using a fake backend (no model download needed). |
| UPDATED | `main.py` | Now takes --stages (ingestion, asr) so Phases 1 and 2 run from one command. |
| UPDATED | `src/common/utils.py` | CONFIG_SCHEMA extended with the asr and asr_gates sections. |
| UPDATED | `src/common/base_stage.py` | New config_keys: a stage's cache hashes only the sections it uses. |
| UPDATED | `requirements.txt` | Adds faster-whisper, jiwer, srt. |
| UPDATED | `env_check.py` | Checks faster_whisper, jiwer and srt as well. |

## Also changed (one line, not listed in full)

`src/ingestion/video_ingestion.py`: added `config_keys = ["ingestion", "gates", "seed"]` to VideoIngestionStage, so editing the asr section no longer forces ingestion to re-run.

## Quick start

```
pip install -r requirements.txt
python -m pytest -q tests                                    # 38 passed
python main.py --config configs/phase2.yaml --video clip.mp4 --video-id clip_001 --stages ingestion asr
```

```
python main.py --config configs/phase2.yaml --video-id clip_001 --stages asr     # Phase 1 already done
```

## Ground truth for evaluation

Create `data/subtitles/clip_001.gt.json` by hand (all parts optional). Without it the stage checks structure only and says so in the metrics.

```
{
  "reference_text": "hand-corrected transcript of the clip",
  "timestamp_marks": [{"text": "where are you going", "start": 12.40}],
  "non_speech": [[0.0, 3.2], [50.0, 58.5]]
}
```

## Phase 2 gate (suggested starting values, set in configs/phase2.yaml)

| Metric | Gate | How computed |
|---|---|---|
| WER | <= 15% (clean dialogue) | jiwer after lowercasing and removing punctuation |
| Start-time MAE | <= 0.5 s | 30 manually marked utterances, paired to segments by text similarity |
| Hallucination rate | <= 5% | segments mostly inside manually marked non-speech regions |
| Timeline | always | ids sequential, end > start, sorted, no overlaps, no empty text |
| Real-time factor | reported | processing time / audio duration; use the model-size study |

Known limits: English is assumed when words are joined with spaces (set asr.language and check other languages); merging fragments can join two speakers' words across a Whisper segment boundary, which Phase 3 handles with its 'mixed' label; the thresholds are starting points, not published standards.

## 1. configs/phase2.yaml [NEW]

Phase 1 settings plus the new asr and asr_gates sections (all thresholds live here).

```yaml
# Phase 1 + Phase 2 settings (ingestion, then speech recognition).
# Every threshold lives here; nothing is hard-coded in src/.
data_dir: data
results_dir: results
log_dir: logs
seed: 42

ingestion:
  max_duration_s: 300        # development cap; 0 = no limit
  min_duration_s: 1.0        # reject clips shorter than this
  audio_sample_rate: 16000   # Whisper / pyannote expect 16 kHz mono
  force_cfr: true            # re-encode variable frame rate to constant frame rate
  vfr_tolerance: 0.02        # relative gap between r_frame_rate and avg_frame_rate
  random_frame_checks: 20    # number of random seek-and-read checks
  black_frame_mean: 8.0      # mean pixel value (0-255) below which a frame counts as black
  max_bad_frame_fraction: 0.1  # tolerated share of black/unreadable sampled frames
  x264_crf: 18               # quality used only when a re-encode is needed
  x264_preset: veryfast

gates:                       # Phase 1 pass/fail thresholds (Section 5, Phase 1)
  frame_count_rel_diff: 0.01 # OpenCV vs ffprobe frame count, relative
  audio_video_diff_s: 0.1    # |audio duration - video duration|
  decode_errors: 0           # lines printed by the ffmpeg null decode

asr:                          # Phase 2 - Whisper speech recognition
  backend: faster-whisper     # faster-whisper | openai-whisper
  model_size: small           # tiny | base | small | medium | large-v3 (see model-size study)
  language: en                # null = auto-detect
  device: auto                # auto | cpu | cuda
  compute_type: auto          # faster-whisper only: auto | int8 | float16 | float32
  vad_filter: true            # Silero VAD (faster-whisper only): suppresses silence hallucination
  beam_size: 5
  condition_on_previous_text: false   # false reduces repetition loops / runaway hallucination
  # --- filtering ---
  max_no_speech_prob: 0.6     # drop a segment when no_speech_prob > this AND ...
  min_avg_logprob: -1.0       # ... avg_logprob < this (Whisper's own heuristic)
  hallucination_phrases:      # known silence hallucinations (matched on short segments)
    - thanks for watching
    - thank you for watching
    - please subscribe
    - subtitles by
    - amara.org
    - like and subscribe
  # --- balloon-sized segmentation ---
  max_words: 12               # split segments longer than this ...
  max_duration_s: 4.0         # ... or longer than this
  split_gap_s: 0.8            # always split at a pause this long inside a segment
  min_duration_s: 0.5         # merge fragments shorter than this
  merge_gap_s: 0.5            # ... only with a neighbour closer than this
  merge_max_words: 16         # ... and only if the merged line stays this short
  # --- evaluation ---
  timestamp_match_min_ratio: 0.5   # text similarity needed to pair a manual mark with a segment
  nonspeech_overlap_ratio: 0.5     # segment counts as hallucination if this much lies in silence

asr_gates:                    # suggested starting thresholds - tune after your first run
  wer_max: 0.15               # clean dialogue
  start_mae_s_max: 0.5
  hallucination_rate_max: 0.05
```

## 2. src/audio/postprocess.py [NEW]

Pure functions: filter hallucinations, split to balloon size, merge short fragments, fix the timeline.

```
"""Turn raw Whisper output into clean, balloon-sized, time-ordered segments.

Pure functions (no model, no files) so they are easy to unit-test.

Raw segment format (what every backend must return):
  {"start": float, "end": float, "text": str, "avg_logprob": float,
   "no_speech_prob": float, "words": [{"w": str, "start": float, "end": float, "p": float}]}
"""
from __future__ import annotations

import re

STRONG = (".", "!", "?", "\u2026")            # sentence end
SOFT = (",", ";", ":", "-", "\u2013", "\u2014")   # clause break
_TRAIL = "\"')]}"


def _norm(text: str) -> str:
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9 ]", " ", text.lower())).strip()


def _ends_with(word: dict, marks: tuple) -> bool:
    return word["w"].rstrip(_TRAIL).endswith(marks)


def _dur(seg: dict) -> float:
    return seg["end"] - seg["start"]


# ---- 1. filter ------------------------------------------------------------
def filter_segments(raw: list[dict], acfg: dict) -> tuple[list[dict], list[dict]]:
    """Split raw segments into (kept, dropped). Dropped ones carry a 'reason'."""
    phrases = [_norm(p) for p in acfg["hallucination_phrases"]]
    kept, dropped = [], []
    for seg in raw:
        text = seg["text"].strip()
        norm = _norm(text)
        reason = None
        if not norm:
            reason = "empty"
        elif (seg.get("no_speech_prob", 0.0) > acfg["max_no_speech_prob"]
              and seg.get("avg_logprob", 0.0) < acfg["min_avg_logprob"]):
            reason = "no_speech_low_confidence"
        else:
            for p in phrases:
                # only short segments: a real sentence may contain the phrase
                if p and p in norm and len(norm.split()) <= len(p.split()) + 3:
                    reason = f"hallucination_phrase:{p}"
                    break
        if reason:
            dropped.append({"start": round(seg["start"], 3), "end": round(seg["end"], 3),
                            "text": text, "reason": reason})
        else:
            kept.append(seg)
    return kept, dropped


# ---- 2. words -------------------------------------------------------------
def ensure_words(seg: dict) -> dict:
    """If the backend gave no word timestamps, spread the words evenly over the segment."""
    if seg.get("words"):
        return seg
    tokens = seg["text"].split()
    step = _dur(seg) / max(len(tokens), 1)
    seg = dict(seg)
    seg["words"] = [{"w": t, "start": seg["start"] + i * step,
                     "end": seg["start"] + (i + 1) * step, "p": None}
                    for i, t in enumerate(tokens)]
    seg["words_estimated"] = True
    return seg


# ---- 3. split -------------------------------------------------------------
def _sentence_split(words: list[dict], gap: float) -> list[list[dict]]:
    chunks, cur = [], []
    for i, w in enumerate(words):
        cur.append(w)
        last = i == len(words) - 1
        pause = 0.0 if last else words[i + 1]["start"] - w["end"]
        if last or _ends_with(w, STRONG) or pause >= gap:
            chunks.append(cur)
            cur = []
```

```
    return chunks


def _too_big(chunk: list[dict], acfg: dict) -> bool:
    return len(chunk) > acfg["max_words"] or \
        (chunk[-1]["end"] - chunk[0]["start"]) > acfg["max_duration_s"]


def _split_balanced(chunk: list[dict], acfg: dict) -> list[list[dict]]:
    """Recursively cut an oversized chunk near its middle, preferring commas, then pauses."""
    n = len(chunk)
    if n < 2 or not _too_big(chunk, acfg):
        return [chunk]
    lo = 2 if n >= 4 else 1                      # avoid one-word pieces when possible
    hi = n - lo
    mid = n / 2
    soft = [i for i in range(lo, hi + 1) if _ends_with(chunk[i - 1], SOFT)]
    if soft:
        cut = min(soft, key=lambda i: abs(i - mid))
    else:
        lo_g, hi_g = max(lo, n // 3), min(hi, 2 * n // 3)
        rng = range(lo_g, hi_g + 1) if lo_g <= hi_g else range(lo, hi + 1)
        cut = max(rng, key=lambda i: (chunk[i]["start"] - chunk[i - 1]["end"], -abs(i - mid)))
    return _split_balanced(chunk[:cut], acfg) + _split_balanced(chunk[cut:], acfg)


def _make_segment(words: list[dict], parent: dict) -> dict:
    return {"start": words[0]["start"], "end": words[-1]["end"],
            "text": " ".join(w["w"] for w in words),
            "words": words,
            "avg_logprob": parent.get("avg_logprob"),
            "no_speech_prob": parent.get("no_speech_prob")}


def split_segment(seg: dict, acfg: dict) -> list[dict]:
    seg = ensure_words(seg)
    words = [dict(w, w=w["w"].strip()) for w in seg["words"] if w["w"].strip()]
    if not words:
        return []
    out = []
    for sentence in _sentence_split(words, acfg["split_gap_s"]):
        for piece in _split_balanced(sentence, acfg):
            out.append(_make_segment(piece, seg))
    return out


# ---- 4. merge ---------------------------------------------------------------
def _merge_two(a: dict, b: dict) -> dict:
    na, nb = len(a["words"]), len(b["words"])

    def wavg(key):
        va, vb = a.get(key), b.get(key)
        if va is None or vb is None:
            return va if vb is None else vb
        return (va * na + vb * nb) / (na + nb)

    words = a["words"] + b["words"]
    return {"start": a["start"], "end": b["end"], "text": " ".join(w["w"] for w in words),
            "words": words, "avg_logprob": wavg("avg_logprob"),
            "no_speech_prob": wavg("no_speech_prob")}


def merge_short(segs: list[dict], acfg: dict) -> list[dict]:
    """Merge fragments shorter than min_duration_s into the closest neighbour."""
    segs = list(segs)
    changed = True
    while changed:
        changed = False
        for i, s in enumerate(segs):
            if _dur(s) >= acfg["min_duration_s"]:
                continue
            options = []
            if i > 0:
                p = segs[i - 1]
                gap = s["start"] - p["end"]
                if gap <= acfg["merge_gap_s"] and \
                        len(p["words"]) + len(s["words"]) <= acfg["merge_max_words"]:
                    options.append((gap, "prev"))
            if i + 1 < len(segs):
                n = segs[i + 1]
                gap = n["start"] - s["end"]
                if gap <= acfg["merge_gap_s"] and \
                        len(n["words"]) + len(s["words"]) <= acfg["merge_max_words"]:
                    options.append((gap, "next"))
            if not options:
                continue
```

```
            side = min(options)[1]
            if side == "prev":
                segs[i - 1:i + 1] = [_merge_two(segs[i - 1], s)]
            else:
                segs[i:i + 2] = [_merge_two(s, segs[i + 1])]
            changed = True
            break
    return segs


# ---- 5. timeline ----------------------------------------------------------------
MIN_SEG_S = 0.05


def enforce_timeline(segs: list[dict], audio_duration: float | None = None) -> list[dict]:
    """Sort, remove overlaps, clamp to the audio length, drop zero-length segments."""
    segs = sorted((dict(s) for s in segs), key=lambda s: (s["start"], s["end"]))
    out: list[dict] = []
    for s in segs:
        if audio_duration is not None:
            s["start"] = min(max(s["start"], 0.0), audio_duration)
            s["end"] = min(s["end"], audio_duration)
        if out and s["start"] < out[-1]["end"]:
            prev = out[-1]
            if s["start"] - prev["start"] >= MIN_SEG_S:
                prev["end"] = s["start"]            # trim the earlier one
            else:
                s["start"] = prev["end"]            # or push this one later
        if s["end"] - s["start"] < MIN_SEG_S:
            continue
        out.append(s)
    return out


# ---- 6. full pipeline -----------------------------------------------------------
def postprocess(raw: list[dict], acfg: dict,
                audio_duration: float | None = None) -> tuple[list[dict], list[dict]]:
    """raw Whisper segments -> (final segments with segment_id, dropped segments)."""
    kept, dropped = filter_segments(raw, acfg)
    pieces: list[dict] = []
    for seg in sorted(kept, key=lambda s: s["start"]):
        pieces.extend(split_segment(seg, acfg))
    merged = merge_short(pieces, acfg)
    final = enforce_timeline(merged, audio_duration)
    result = []
    for i, s in enumerate(final, 1):
        result.append({
            "segment_id": i,
            "start": round(s["start"], 3),
            "end": round(s["end"], 3),
            "text": s["text"].strip(),
            "words": [{"w": w["w"], "start": round(w["start"], 3), "end": round(w["end"], 3)}
                      for w in s["words"]],
            "avg_logprob": None if s["avg_logprob"] is None else round(s["avg_logprob"], 3),
            "no_speech_prob": None if s["no_speech_prob"] is None
            else round(s["no_speech_prob"], 3),
        })
    return result, dropped


def validate_timeline(segs: list[dict]) -> list[str]:
    """Return a list of problems (empty list = OK). Used by the stage gate and tests."""
    problems = []
    for i, s in enumerate(segs):
        if s["segment_id"] != i + 1:
            problems.append(f"segment_id {s['segment_id']} out of sequence at position {i + 1}")
        if s["end"] <= s["start"]:
            problems.append(f"segment {s['segment_id']}: end <= start")
        if i and s["start"] < segs[i - 1]["start"]:
            problems.append(f"segment {s['segment_id']}: not sorted by start")
        if i and s["start"] < segs[i - 1]["end"] - 1e-3:
            problems.append(f"segment {s['segment_id']}: overlaps the previous segment")
        if not s["text"].strip():
            problems.append(f"segment {s['segment_id']}: empty text")
    return problems
```

## 3. src/audio/srt_io.py [NEW]

Read and write SRT files with the srt library.

```
"""Read and write SRT subtitle files (uses the `srt` library)."""
from __future__ import annotations

from datetime import timedelta
from pathlib import Path

import srt


def write_srt(segments: list[dict], path: str | Path) -> None:
    subs = [srt.Subtitle(index=s["segment_id"], start=timedelta(seconds=s["start"]),
                         end=timedelta(seconds=s["end"]), content=s["text"])
            for s in segments]
    Path(path).write_text(srt.compose(subs), encoding="utf-8")


def read_srt(path: str | Path) -> list[dict]:
    text = Path(path).read_text(encoding="utf-8")
    return [{"segment_id": sub.index, "start": sub.start.total_seconds(),
             "end": sub.end.total_seconds(), "text": sub.content}
            for sub in srt.parse(text)]
```

## 4. src/audio/whisper_asr.py [NEW]

Whisper backends (faster-whisper, openai-whisper), run_asr() and the AsrStage.

```
"""Phase 2: speech recognition with Whisper -> timestamped, balloon-sized dialogue segments.

Reads  : data/metadata/{id}.json (Phase 1) and the 16 kHz WAV it points to.
Writes : data/subtitles/{id}.json and data/subtitles/{id}.srt
Metrics: data/subtitles/{id}.metrics.json
"""
from __future__ import annotations

import json
import time
from datetime import datetime
from pathlib import Path

from src.audio.postprocess import postprocess, validate_timeline
from src.audio.srt_io import write_srt
from src.common.base_stage import BaseStage
from src.common.utils import artifact_path
from src.evaluation.asr_metrics import evaluate_against_gt


class AsrError(RuntimeError):
    pass


# ---- backends ---------------------------------------------------------------
# A backend is any function (wav_path, acfg, log) -> (raw_segments, info_dict).
_MODEL_CACHE: dict = {}


def _transcribe_faster_whisper(wav: str, acfg: dict, log=None):
    try:
        from faster_whisper import WhisperModel
    except ImportError as exc:
        raise AsrError("faster-whisper is not installed (pip install faster-whisper)") from exc
    key = ("fw", acfg["model_size"], acfg["device"], acfg["compute_type"])
    if key not in _MODEL_CACHE:
        if log:
            log.info("loading faster-whisper model '%s'", acfg["model_size"])
        try:
            _MODEL_CACHE[key] = WhisperModel(acfg["model_size"], device=acfg["device"],
                                             compute_type=acfg["compute_type"])
        except Exception as exc:                    # noqa: BLE001 (download / device errors)
            raise AsrError(f"could not load faster-whisper model '{acfg['model_size']}': "
                           f"{type(exc).__name__}: {str(exc)[:200]}. Check your internet "
                           "connection (first run downloads the model) or device settings."
                           ) from exc
    model = _MODEL_CACHE[key]
    segments, info = model.transcribe(
        wav, language=acfg["language"], beam_size=acfg["beam_size"], word_timestamps=True,
        vad_filter=acfg["vad_filter"],
        condition_on_previous_text=acfg["condition_on_previous_text"])
    raw = []
    for seg in segments:                        # generator: decoding happens here
        raw.append({
            "start": float(seg.start), "end": float(seg.end), "text": seg.text,
            "avg_logprob": float(seg.avg_logprob), "no_speech_prob": float(seg.no_speech_prob),
            "words": [{"w": w.word, "start": float(w.start), "end": float(w.end),
                       "p": float(w.probability)} for w in (seg.words or [])],
        })
    return raw, {"language": info.language, "language_probability": info.language_probability}


def _transcribe_openai_whisper(wav: str, acfg: dict, log=None):
    try:
        import whisper
    except ImportError as exc:
        raise AsrError("openai-whisper is not installed (pip install openai-whisper)") from exc
    device = None if acfg["device"] == "auto" else acfg["device"]
    key = ("ow", acfg["model_size"], device)
    if key not in _MODEL_CACHE:
        if log:
            log.info("loading openai-whisper model '%s'", acfg["model_size"])
        try:
            _MODEL_CACHE[key] = whisper.load_model(acfg["model_size"], device=device)
        except Exception as exc:                    # noqa: BLE001
            raise AsrError(f"could not load openai-whisper model '{acfg['model_size']}': "
                           f"{type(exc).__name__}: {str(exc)[:200]}") from exc
    model = _MODEL_CACHE[key]
    if acfg["vad_filter"] and log:
        log.warning("vad_filter is only supported by faster-whisper; ignored")
    result = model.transcribe(
        wav, language=acfg["language"], word_timestamps=True,
```

```
        condition_on_previous_text=acfg["condition_on_previous_text"],
        beam_size=acfg["beam_size"], fp16=(model.device.type == "cuda"))
    raw = [{
        "start": float(s["start"]), "end": float(s["end"]), "text": s["text"],
        "avg_logprob": float(s["avg_logprob"]), "no_speech_prob": float(s["no_speech_prob"]),
        "words": [{"w": w["word"], "start": float(w["start"]), "end": float(w["end"]),
                   "p": float(w.get("probability", 1.0))} for w in s.get("words", [])],
    } for s in result["segments"]]
    return raw, {"language": result.get("language")}


BACKENDS = {"faster-whisper": _transcribe_faster_whisper,
            "openai-whisper": _transcribe_openai_whisper}


def run_asr(acfg: dict, wav_path: str, audio_duration: float, log=None, transcribe_fn=None):
    """Transcribe + post-process. Returns (segments, dropped, info, runtime_s)."""
    fn = transcribe_fn or BACKENDS.get(acfg["backend"])
    if fn is None:
        raise AsrError(f"Unknown asr.backend '{acfg['backend']}' (use {list(BACKENDS)})")
    t0 = time.time()
    raw, info = fn(str(wav_path), acfg, log)
    runtime = time.time() - t0
    segments, dropped = postprocess(raw, acfg, audio_duration)
    return segments, dropped, info, runtime


# ---- stage ------------------------------------------------------------------------
class AsrStage(BaseStage):
    name = "asr"
    requires = ["ingestion"]
    config_keys = ["asr", "asr_gates", "seed"]

    def __init__(self, cfg, video_id, logger=None, transcribe_fn=None):
        super().__init__(cfg, video_id, logger)
        self.transcribe_fn = transcribe_fn
        self._artifact = artifact_path(cfg, "subtitles", video_id, "json")
        self.srt_path = artifact_path(cfg, "subtitles", video_id, "srt")
        self.gt_path = artifact_path(cfg, "subtitles", video_id, "gt.json")

    @property
    def artifact(self) -> Path:
        return self._artifact

    def run(self) -> None:
        meta_path = artifact_path(self.cfg, "metadata", self.video_id, "json")
        if not meta_path.exists():
            raise AsrError(f"{meta_path} not found: run Phase 1 (ingestion) first")
        meta = json.loads(meta_path.read_text(encoding="utf-8"))
        acfg = self.cfg["asr"]

        if not meta["has_audio"]:
            if self.log:
                self.log.info("no audio track: writing an empty transcript")
            segments, dropped, info, runtime = [], [], {"language": None}, 0.0
        else:
            segments, dropped, info, runtime = run_asr(
                acfg, meta["audio_path"], meta["audio_duration_s"], self.log,
                self.transcribe_fn)

        audio_dur = meta.get("audio_duration_s") or 0.0
        payload = {
            "video_id": self.video_id,
            "model": acfg["model_size"],
            "backend": acfg["backend"],
            "language": info.get("language"),
            "audio_duration_s": audio_dur,
            "runtime_s": round(runtime, 3),
            "rtf": round(runtime / audio_dur, 4) if audio_dur else None,
            "n_dropped": len(dropped),
            "segments": segments,
            "dropped": dropped,
            "created": datetime.now().isoformat(timespec="seconds"),
        }
        self.artifact.write_text(json.dumps(payload, indent=2, ensure_ascii=False),
                                 encoding="utf-8")
        write_srt(segments, self.srt_path)

    def evaluate(self, gt=None) -> dict:
        data = json.loads(self.artifact.read_text(encoding="utf-8"))
        segments = data["segments"]
        metrics = {
            "model": data["model"],
            "n_segments": len(segments),
            "n_dropped": data["n_dropped"],
            "rtf": data["rtf"],
```

```
            "timeline_problems": validate_timeline(segments),
            "mean_words_per_segment": round(
                sum(len(s["words"]) for s in segments) / len(segments), 2) if segments else 0,
        }
        if gt is None and self.gt_path.exists():
            gt = json.loads(self.gt_path.read_text(encoding="utf-8"))
        metrics["ground_truth"] = bool(gt)
        if gt:
            metrics.update(evaluate_against_gt(segments, gt, self.cfg["asr"]))
        return metrics

    def gate(self, metrics: dict) -> tuple[bool, str]:
        g = self.cfg["asr_gates"]
        fails = list(metrics["timeline_problems"][:3])
        if metrics.get("wer") is not None and metrics["wer"] > g["wer_max"]:
            fails.append(f"WER {metrics['wer']:.3f} > {g['wer_max']}")
        mae = metrics.get("start_mae_s")
        if mae is not None and mae > g["start_mae_s_max"]:
            fails.append(f"start-time MAE {mae}s > {g['start_mae_s_max']}s")
        hr = metrics.get("hallucination_rate")
        if hr is not None and hr > g["hallucination_rate_max"]:
            fails.append(f"hallucination rate {hr} > {g['hallucination_rate_max']}")
        if fails:
            return False, "; ".join(fails)
        if not metrics["ground_truth"]:
            return True, "structure OK; no ground truth file (WER/MAE/hallucination not evaluated)"
        return True, "all checks passed"
```

## 5. src/evaluation/asr_metrics.py [NEW]

WER, CER, start-time MAE, hallucination rate, ground-truth evaluation.

```
"""Phase 2 evaluation: WER, CER, start-time error, hallucination rate.

Ground-truth file (data/subtitles/{video_id}.gt.json), all parts optional:
{
  "reference_text": "full hand-corrected transcript of the clip",
  "timestamp_marks": [{"text": "where are you going", "start": 12.40}],
  "non_speech": [[0.0, 3.2], [50.0, 58.5]]        # regions with no speech (music, silence)
}
"""
from __future__ import annotations

import difflib
import re


def normalise_text(text: str) -> str:
    """Lowercase, drop punctuation (apostrophes removed), collapse whitespace."""
    text = text.lower().replace("'", "").replace("\u2019", "")
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9 ]", " ", text)).strip()


def wer_cer(reference: str, hypothesis: str) -> dict:
    """WER and CER via jiwer after normalisation."""
    try:
        import jiwer
    except ImportError as exc:
        raise RuntimeError("jiwer is not installed (pip install jiwer)") from exc
    ref, hyp = normalise_text(reference), normalise_text(hypothesis)
    if not ref:
        raise ValueError("reference transcript is empty after normalisation")
    if not hyp:
        return {"wer": 1.0, "cer": 1.0, "ref_words": len(ref.split())}
    return {"wer": round(jiwer.wer(ref, hyp), 4), "cer": round(jiwer.cer(ref, hyp), 4),
            "ref_words": len(ref.split())}


def start_time_mae(segments: list[dict], marks: list[dict], min_ratio: float = 0.5) -> dict:
    """Mean absolute error of segment start vs manual marks.

    Each mark is paired with the segment whose text is most similar; marks whose best
    similarity is below min_ratio count as unmatched (reported, not averaged).
    """
    errors, unmatched = [], 0
    norm_segs = [normalise_text(s["text"]) for s in segments]
    for mark in marks:
        target = normalise_text(mark["text"])
        best, best_ratio = None, 0.0
        for seg, ns in zip(segments, norm_segs):
            ratio = difflib.SequenceMatcher(None, target, ns).ratio()
            if ratio > best_ratio:
                best, best_ratio = seg, ratio
        if best is None or best_ratio < min_ratio:
            unmatched += 1
        else:
            errors.append(abs(best["start"] - mark["start"]))
    mae = round(sum(errors) / len(errors), 4) if errors else None
    return {"start_mae_s": mae, "marks_matched": len(errors), "marks_unmatched": unmatched}


def hallucination_rate(segments: list[dict], non_speech: list[list[float]],
                       min_overlap: float = 0.5) -> dict:
    """Share of segments lying mostly (>= min_overlap of their length) in non-speech regions."""
    if not segments:
        return {"hallucination_rate": 0.0, "hallucinated": 0}
    bad = 0
    for s in segments:
        length = max(s["end"] - s["start"], 1e-6)
        overlap = sum(max(0.0, min(s["end"], b) - max(s["start"], a)) for a, b in non_speech)
        if overlap / length >= min_overlap:
            bad += 1
    return {"hallucination_rate": round(bad / len(segments), 4), "hallucinated": bad}


def evaluate_against_gt(segments: list[dict], gt: dict, acfg: dict) -> dict:
    """Compute every metric the ground-truth file allows."""
    metrics: dict = {}
    if gt.get("reference_text"):
        hyp = " ".join(s["text"] for s in segments)
        metrics.update(wer_cer(gt["reference_text"], hyp))
    if gt.get("timestamp_marks"):
        metrics.update(start_time_mae(segments, gt["timestamp_marks"],
                                      acfg["timestamp_match_min_ratio"]))
```

```
    if "non_speech" in gt:
        metrics.update(hallucination_rate(segments, gt["non_speech"],
                                          acfg["nonspeech_overlap_ratio"]))
    return metrics
```

## 6. src/evaluation/asr_model_study.py [NEW]

tiny/base/small/medium study: WER vs real-time factor, writes a CSV.

```
"""Model-size study: WER and real-time factor for tiny/base/small/medium on one labelled clip.

Usage:
  python -m src.evaluation.asr_model_study --config configs/phase2.yaml --video-id clip_001 \
      --sizes tiny base small medium
Needs data/subtitles/{id}.gt.json with a reference_text. Writes results/asr_model_study.csv.
Pick the knee of the WER-vs-RTF curve and justify it in the report.
"""
from __future__ import annotations

import argparse
import copy
import csv
import json
from pathlib import Path

from src.audio.whisper_asr import run_asr
from src.common.utils import artifact_path, load_config, set_seed
from src.evaluation.asr_metrics import evaluate_against_gt


def main(argv=None) -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", required=True)
    ap.add_argument("--video-id", required=True)
    ap.add_argument("--sizes", nargs="+", default=["tiny", "base", "small", "medium"])
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    set_seed(cfg["seed"])
    meta = json.loads(artifact_path(cfg, "metadata", args.video_id).read_text(encoding="utf-8"))
    gt_path = artifact_path(cfg, "subtitles", args.video_id, "gt.json")
    gt = json.loads(gt_path.read_text(encoding="utf-8"))

    rows = []
    for size in args.sizes:
        acfg = copy.deepcopy(cfg["asr"])
        acfg["model_size"] = size
        segs, dropped, _, runtime = run_asr(acfg, meta["audio_path"], meta["audio_duration_s"])
        m = evaluate_against_gt(segs, gt, acfg)
        rows.append({"model": size, "wer": m.get("wer"), "cer": m.get("cer"),
                     "start_mae_s": m.get("start_mae_s"),
                     "hallucination_rate": m.get("hallucination_rate"),
                     "rtf": round(runtime / meta["audio_duration_s"], 4),
                     "segments": len(segs), "dropped": len(dropped)})
        print(rows[-1])

    out = Path(cfg["results_dir"]) / "asr_model_study.csv"
    out.parent.mkdir(parents=True, exist_ok=True)
    with open(out, "w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    print("wrote", out)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

## 7. tests/test_phase2.py [NEW]

23 new unit tests using a fake backend (no model download needed).

```
"""Phase 2 unit tests. A fake backend is used, so no Whisper model download is needed."""
from __future__ import annotations

import json
from pathlib import Path

import pytest

from src.audio import postprocess as pp
from src.audio.srt_io import read_srt
from src.audio.whisper_asr import AsrError, AsrStage
from src.common.utils import load_config
from src.evaluation import asr_metrics as am
from tests.conftest import ROOT, needs_ffmpeg


# ---- helpers ---------------------------------------------------------------------------
def words(text, start, step=0.3):
    out, t = [], start
    for tok in text.split():
        out.append({"w": " " + tok, "start": t, "end": t + step * 0.9, "p": 0.9})
        t += step
    return out


def raw_seg(text, start, step=0.3, lp=-0.3, ns=0.02):
    w = words(text, start, step)
    return {"start": w[0]["start"], "end": w[-1]["end"], "text": " " + text,
            "avg_logprob": lp, "no_speech_prob": ns, "words": w}


@pytest.fixture
def acfg():
    return load_config(ROOT / "configs" / "phase2.yaml")["asr"]


@pytest.fixture
def cfg2(tmp_path):
    c = load_config(ROOT / "configs" / "phase2.yaml")
    c["data_dir"] = str(tmp_path / "data")
    c["results_dir"] = str(tmp_path / "results")
    c["log_dir"] = str(tmp_path / "logs")
    c["ingestion"]["random_frame_checks"] = 10
    return c


def fake_backend(raw):
    def fn(wav, acfg, log=None):
        assert Path(wav).exists()
        return list(raw), {"language": "en"}
    return fn


# ---- config ----------------------------------------------------------------------------
def test_phase2_config_loads(acfg):
    assert acfg["max_words"] == 12 and acfg["model_size"] == "small"


# ---- post-processing -------------------------------------------------------------------
def test_long_segment_is_split_to_balloon_size(acfg):
    text = ("Well I think we should leave now, because the storm is coming and "
            "nobody wants to be here when it arrives at the old harbour tonight")
    segs, _ = pp.postprocess([raw_seg(text, 10.0)], acfg)
    assert len(segs) >= 2
    for s in segs:
        assert len(s["words"]) <= acfg["max_words"]
        assert s["end"] - s["start"] <= acfg["max_duration_s"] + 1e-6
    joined = " ".join(w["w"] for s in segs for w in s["words"])
    assert joined == " ".join(text.split())                    # no word lost or reordered


def test_split_prefers_comma_and_never_makes_one_word_pieces(acfg):
    text = "If you really want to know the truth, then listen very carefully to me"
    segs, _ = pp.postprocess([raw_seg(text, 0.0, step=0.25)], acfg)
    assert segs[0]["text"].endswith("truth,")
    assert all(len(s["words"]) >= 2 for s in segs)


def test_split_at_sentence_end_and_long_pause(acfg):
    a = raw_seg("Where are you going?", 1.0)
    w2 = words("To the harbour.", a["end"] + 0.1)
    seg = dict(a, words=a["words"] + w2, end=w2[-1]["end"], text=a["text"] + " To the harbour.")
```

```
    segs, _ = pp.postprocess([seg], acfg)
    assert [s["text"] for s in segs] == ["Where are you going?", "To the harbour."]

    gap = raw_seg("Hold on", 1.0)
    later = words("one more thing", gap["end"] + 1.5)
    seg = dict(gap, words=gap["words"] + later, end=later[-1]["end"], text="Hold on one more thing")
    segs, _ = pp.postprocess([seg], acfg)
    assert len(segs) == 2


def test_short_fragments_are_merged(acfg):
    raw = [raw_seg("Hey", 5.0, step=0.3),                          # 0.27 s fragment
           raw_seg("wait for me", 5.4, step=0.3)]
    segs, _ = pp.postprocess(raw, acfg)
    assert len(segs) == 1 and segs[0]["text"] == "Hey wait for me"


def test_short_fragment_far_from_neighbours_is_kept(acfg):
    raw = [raw_seg("Hey", 5.0), raw_seg("much later line here", 20.0)]
    segs, _ = pp.postprocess(raw, acfg)
    assert len(segs) == 2


def test_hallucinations_dropped_and_reported(acfg):
    raw = [raw_seg("Thanks for watching", 0.0, ns=0.1, lp=-0.2),    # known phrase
           raw_seg("blah blah blah", 3.0, ns=0.9, lp=-1.8),         # silence + low confidence
           raw_seg("I thanks for watching you every day from afar", 6.0),  # real sentence stays
           raw_seg("Real dialogue stays", 10.0)]
    segs, dropped = pp.postprocess(raw, acfg)
    texts = [s["text"] for s in segs]
    assert "Real dialogue stays" in texts and len(texts) == 2
    reasons = {d["reason"].split(":")[0] for d in dropped}
    assert reasons == {"hallucination_phrase", "no_speech_low_confidence"}


def test_overlapping_segments_become_non_overlapping(acfg):
    a, b = raw_seg("first line of speech here", 1.0, step=0.4), raw_seg("second line now", 2.0)
    segs, _ = pp.postprocess([b, a], acfg, audio_duration=30.0)   # also given out of order
    assert pp.validate_timeline(segs) == []
    assert segs[0]["text"].startswith("first")


def test_segments_clamped_to_audio_duration(acfg):
    segs, _ = pp.postprocess([raw_seg("this runs past the end of audio", 9.0)], acfg,
                             audio_duration=10.0)
    assert segs and all(s["end"] <= 10.0 for s in segs)


def test_missing_word_timestamps_are_estimated(acfg):
    seg = {"start": 2.0, "end": 4.0, "text": " no word times here", "avg_logprob": -0.2,
           "no_speech_prob": 0.01, "words": []}
    segs, _ = pp.postprocess([seg], acfg)
    assert segs[0]["text"] == "no word times here" and len(segs[0]["words"]) == 4


def test_segment_ids_sequential_from_one(acfg):
    raw = [raw_seg(f"line number {i} is here", 10.0 * i) for i in range(1, 5)]
    segs, _ = pp.postprocess(raw, acfg)
    assert [s["segment_id"] for s in segs] == list(range(1, len(segs) + 1))


def test_validate_timeline_detects_problems():
    bad = [{"segment_id": 1, "start": 2.0, "end": 1.0, "text": "x"},
           {"segment_id": 3, "start": 0.5, "end": 3.0, "text": ""}]
    assert len(pp.validate_timeline(bad)) >= 4


# ---- metrics ---------------------------------------------------------------
def test_wer_cer_known_values():
    m = am.wer_cer("The cat sat.", "the cat sat down")
    assert m["wer"] == 0.3333 and m["ref_words"] == 3          # 1 insertion / 3 words
    assert am.wer_cer("Don't go!", "dont go")["wer"] == 0.0     # punctuation + apostrophes ignored
    assert am.wer_cer("hello world", "")["wer"] == 1.0


def test_start_time_mae_pairs_by_text():
    segs = [{"text": "Where are you going?", "start": 12.5},
            {"text": "To the harbour.", "start": 15.0}]
    m = am.start_time_mae(segs, [{"text": "where are you going", "start": 12.4},
                                 {"text": "to the harbour", "start": 15.3},
                                 {"text": "completely different words", "start": 40.0}])
    assert m["marks_matched"] == 2 and m["marks_unmatched"] == 1
    assert m["start_mae_s"] == pytest.approx(0.2, abs=1e-3)


def test_hallucination_rate_uses_non_speech_regions():
```

```
    segs = [{"start": 0.0, "end": 2.0}, {"start": 5.0, "end": 7.0}, {"start": 9.0, "end": 10.0}]
    m = am.hallucination_rate(segs, [[0.0, 3.0]], 0.5)
    assert m["hallucinated"] == 1 and m["hallucination_rate"] == pytest.approx(1 / 3, abs=1e-3)


# ---- SRT ---------------------------------------------------------------------------
def test_srt_round_trips(acfg, tmp_path):
    from src.audio.srt_io import write_srt
    segs, _ = pp.postprocess([raw_seg("Where are you going?", 12.42),
                              raw_seg("To the harbour.", 15.0)], acfg)
    path = tmp_path / "x.srt"
    write_srt(segs, path)
    back = read_srt(path)
    assert [b["text"] for b in back] == [s["text"] for s in segs]
    for b, s in zip(back, segs):
        assert abs(b["start"] - s["start"]) < 1e-3 and abs(b["end"] - s["end"]) < 1e-3


# ---- stage end to end (fake backend) -----------------------------------------------
@needs_ffmpeg
class TestStage:
    @pytest.fixture
    def ingested(self, cfg2, make):
        import logging
        from src.common.utils import close_logger
        from src.ingestion.video_ingestion import ingest
        ok, vid = ingest(cfg2, str(make(duration=20)), "t002")
        close_logger(logging.getLogger("cinecomic.t002"))
        assert ok
        return vid

    RAW = [raw_seg("Where are you going?", 2.0), raw_seg("To the harbour.", 5.0),
           raw_seg("Thanks for watching", 9.0)]

    def test_stage_writes_artifacts_and_passes(self, cfg2, ingested):
        stage = AsrStage(cfg2, ingested, transcribe_fn=fake_backend(self.RAW))
        assert stage.execute() is True
        data = json.loads(stage.artifact.read_text())
        assert data["video_id"] == ingested and data["model"] == "small"
        assert [s["text"] for s in data["segments"]] == ["Where are you going?", "To the harbour."]
        assert data["n_dropped"] == 1
        assert set(data["segments"][0]) >= {"segment_id", "start", "end", "text", "words",
                                            "avg_logprob", "no_speech_prob"}
        assert len(read_srt(stage.srt_path)) == 2
        metrics = json.loads(stage.metrics_path.read_text())
        assert metrics["passed"] and metrics["ground_truth"] is False

    def test_gate_passes_with_good_ground_truth(self, cfg2, ingested):
        stage = AsrStage(cfg2, ingested, transcribe_fn=fake_backend(self.RAW))
        gt = {"reference_text": "Where are you going? To the harbour.",
              "timestamp_marks": [{"text": "where are you going", "start": 2.1}],
              "non_speech": [[8.0, 12.0]]}
        stage.gt_path.write_text(json.dumps(gt))
        assert stage.execute() is True
        m = json.loads(stage.metrics_path.read_text())
        assert m["wer"] == 0.0 and m["hallucination_rate"] == 0.0

    def test_gate_fails_on_bad_wer_and_hallucination(self, cfg2, ingested):
        raw = [raw_seg("completely wrong words here", 2.0), raw_seg("music lyrics la la la", 9.0)]
        stage = AsrStage(cfg2, ingested, transcribe_fn=fake_backend(raw))
        stage.gt_path.write_text(json.dumps({"reference_text": "Where are you going",
                                             "non_speech": [[8.0, 12.0]]}))
        assert stage.execute() is False
        reason = json.loads(stage.metrics_path.read_text())["reason"]
        assert "WER" in reason and "hallucination" in reason

    def test_cache_and_config_change(self, cfg2, ingested):
        calls = []

        def counting(wav, acfg, log=None):
            calls.append(1)
            return list(self.RAW), {"language": "en"}
        AsrStage(cfg2, ingested, transcribe_fn=counting).execute()
        AsrStage(cfg2, ingested, transcribe_fn=counting).execute()
        assert len(calls) == 1                                  # second run cached
        cfg2["asr"]["max_words"] = 8
        AsrStage(cfg2, ingested, transcribe_fn=counting).execute()
        assert len(calls) == 2                                  # asr config changed -> re-run

    def test_ingestion_cache_unaffected_by_asr_config(self, cfg2, ingested):
        from src.ingestion.video_ingestion import VideoIngestionStage
        cfg2["asr"]["max_words"] = 8
        stage = VideoIngestionStage(cfg2, ingested, Path("x"), "", "x")
        assert stage.is_cached()

    def test_requires_phase1_first(self, cfg2):
```

```
        with pytest.raises(AsrError):
            AsrStage(cfg2, "never_ingested", transcribe_fn=fake_backend([])).run()

    def test_video_without_audio_gives_empty_transcript(self, cfg2, make):
        import logging
        from src.common.utils import close_logger
        from src.ingestion.video_ingestion import ingest
        ok, vid = ingest(cfg2, str(make(audio=False)), "t003")
        close_logger(logging.getLogger("cinecomic.t003"))
        stage = AsrStage(cfg2, vid, transcribe_fn=fake_backend(self.RAW))
        assert stage.execute() is True
        assert json.loads(stage.artifact.read_text())["segments"] == []
```

## 8. main.py [UPDATED]

Now takes --stages (ingestion, asr) so Phases 1 and 2 run from one command.

```
"""CineComic-MML entry point.

Usage:
  # Phase 1 only
  python main.py --config configs/phase1.yaml --video clip.mp4 --video-id clip_001
  # Phase 1 + Phase 2
  python main.py --config configs/phase2.yaml --video clip.mp4 --video-id clip_001 \
      --stages ingestion asr
  # Phase 2 only (Phase 1 already done)
  python main.py --config configs/phase2.yaml --video-id clip_001 --stages asr
Later phases will register their stages in STAGES below.
"""
from __future__ import annotations

import argparse
import sys

from src.common.utils import get_logger, load_config, set_seed
from src.ingestion.metadata import IngestionError
from src.ingestion.video_ingestion import ingest

STAGES = ["ingestion", "asr"]          # extended phase by phase


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="CineComic-MML pipeline")
    ap.add_argument("--config", required=True, help="YAML config, e.g. configs/phase2.yaml")
    ap.add_argument("--video", default=None, help="video file path or URL (needed for ingestion)")
    ap.add_argument("--video-id", default=None,
                    help="id such as clip_001 (default for ingestion: SHA-256 prefix)")
    ap.add_argument("--stages", nargs="+", default=["ingestion"], choices=STAGES,
                    help="stages to run, in order")
    ap.add_argument("--force", action="store_true", help="ignore the cache and re-run")
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    set_seed(cfg["seed"])
    video_id = args.video_id

    if "ingestion" in args.stages:
        if not args.video:
            ap.error("--video is required for the ingestion stage")
        try:
            ok, video_id = ingest(cfg, args.video, video_id, force=args.force)
        except IngestionError as exc:
            print(f"INGESTION FAILED: {exc}", file=sys.stderr)
            return 2
        print(f"[ingestion] video_id={video_id} gate={'PASS' if ok else 'FAIL'}")
        if not ok:
            return 1
    elif not video_id:
        ap.error("--video-id is required when ingestion is not in --stages")

    if "asr" in args.stages:
        from src.audio.whisper_asr import AsrError, AsrStage
        log = get_logger(video_id, cfg["log_dir"])
        try:
            ok = AsrStage(cfg, video_id, log).execute(force=args.force)
        except AsrError as exc:
            print(f"ASR FAILED: {exc}", file=sys.stderr)
            return 2
        print(f"[asr] video_id={video_id} gate={'PASS' if ok else 'FAIL'}")
        if not ok:
            return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

## 9. src/common/utils.py [UPDATED]

CONFIG_SCHEMA extended with the asr and asr_gates sections.

```
"""Phase 0 shared utilities: config loader, seeding, logging, artifact paths."""
from __future__ import annotations

import hashlib
import json
import logging
import random
from pathlib import Path

import yaml


class ConfigError(ValueError):
    """Raised when a config file contains unknown or invalid keys."""


# Allowed keys. A value of None means "any value"; a dict means "nested section".
# Later phases extend this schema when they add their own config sections.
CONFIG_SCHEMA = {
    "data_dir": None,
    "results_dir": None,
    "log_dir": None,
    "seed": None,
    "ingestion": {
        "max_duration_s": None,
        "min_duration_s": None,
        "audio_sample_rate": None,
        "force_cfr": None,
        "vfr_tolerance": None,
        "random_frame_checks": None,
        "black_frame_mean": None,
        "max_bad_frame_fraction": None,
        "x264_crf": None,
        "x264_preset": None,
    },
    "gates": {
        "frame_count_rel_diff": None,
        "audio_video_diff_s": None,
        "decode_errors": None,
    },
    "asr": {
        "backend": None,
        "model_size": None,
        "language": None,
        "device": None,
        "compute_type": None,
        "vad_filter": None,
        "beam_size": None,
        "condition_on_previous_text": None,
        "max_no_speech_prob": None,
        "min_avg_logprob": None,
        "hallucination_phrases": None,
        "max_words": None,
        "max_duration_s": None,
        "split_gap_s": None,
        "min_duration_s": None,
        "merge_gap_s": None,
        "merge_max_words": None,
        "timestamp_match_min_ratio": None,
        "nonspeech_overlap_ratio": None,
    },
    "asr_gates": {
        "wer_max": None,
        "start_mae_s_max": None,
        "hallucination_rate_max": None,
    },
}


def _validate(cfg: dict, schema: dict, prefix: str = "") -> None:
    for key, value in cfg.items():
        path = f"{prefix}{key}"
        if key not in schema:
            raise ConfigError(f"Unknown config key: '{path}'")
        sub = schema[key]
        if isinstance(sub, dict):
            if not isinstance(value, dict):
                raise ConfigError(f"Config key '{path}' must be a mapping")
            _validate(value, sub, prefix=f"{path}.")


def load_config(path: str | Path, schema: dict | None = None) -> dict:
```

```
    """Load a YAML config and reject unknown keys."""
    with open(path, "r", encoding="utf-8") as fh:
        cfg = yaml.safe_load(fh) or {}
    if not isinstance(cfg, dict):
        raise ConfigError("Top level of the config must be a mapping")
    _validate(cfg, schema or CONFIG_SCHEMA)
    return cfg


def config_hash(cfg: dict) -> str:
    """Short stable hash of a config, used for caching and in metrics files."""
    blob = json.dumps(cfg, sort_keys=True, default=str).encode("utf-8")
    return hashlib.sha1(blob).hexdigest()[:8]


def set_seed(seed: int) -> None:
    """Seed Python, NumPy and (if installed) PyTorch."""
    random.seed(seed)
    try:
        import numpy as np
        np.random.seed(seed)
    except ImportError:
        pass
    try:
        import torch
        torch.manual_seed(seed)
    except ImportError:
        pass


def get_logger(video_id: str, log_dir: str | Path = "logs") -> logging.Logger:
    """Logger that writes to the console and to logs/{video_id}.log."""
    logger = logging.getLogger(f"cinecomic.{video_id}")
    if logger.handlers:                     # already configured in this process
        return logger
    logger.setLevel(logging.INFO)
    logger.propagate = False
    fmt = logging.Formatter("%(asctime)s | %(levelname)-7s | %(name)s | %(message)s")
    Path(log_dir).mkdir(parents=True, exist_ok=True)
    file_handler = logging.FileHandler(Path(log_dir) / f"{video_id}.log", encoding="utf-8")
    file_handler.setFormatter(fmt)
    console = logging.StreamHandler()
    console.setFormatter(fmt)
    logger.addHandler(file_handler)
    logger.addHandler(console)
    return logger


def close_logger(logger: logging.Logger) -> None:
    """Release file handles (needed on Windows and in tests)."""
    for handler in list(logger.handlers):
        handler.close()
        logger.removeHandler(handler)


def artifact_path(cfg: dict, kind: str, video_id: str, ext: str = "json") -> Path:
    """Return data/{kind}/{video_id}.{ext} and create the parent folder.

    Example: artifact_path(cfg, "audio", "clip_001", "wav") -> data/audio/clip_001.wav
    """
    path = Path(cfg["data_dir"]) / kind / f"{video_id}.{ext}"
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

## 10. src/common/base_stage.py [UPDATED]

New config_keys: a stage's cache hashes only the sections it uses.

```
"""BaseStage: the contract every phase follows (Section 12.1 of the guide).

input artifacts in -> run() -> one saved artifact -> evaluate() -> gate() ->
metrics JSON saved next to the artifact. Results are cached by config hash.
"""
from __future__ import annotations

import json
import platform
import time
from pathlib import Path

from src.common.utils import config_hash


class BaseStage:
    name = "base"
    requires: list[str] = []            # names of earlier stages
    config_keys: list[str] = []         # config sections that affect this stage's output

    def __init__(self, cfg: dict, video_id: str, logger=None):
        self.cfg = cfg
        self.video_id = video_id
        self.log = logger

    # ---- to be provided by subclasses ----------------------------------
    @property
    def artifact(self) -> Path:
        """Path of the main artifact this stage writes."""
        raise NotImplementedError

    def run(self) -> None:
        raise NotImplementedError

    def evaluate(self, gt=None) -> dict:
        raise NotImplementedError

    def gate(self, metrics: dict) -> tuple[bool, str]:
        raise NotImplementedError

    # ---- shared behaviour ----------------------------------------------
    @property
    def metrics_path(self) -> Path:
        return self.artifact.parent / f"{self.video_id}.metrics.json"

    def cache_key(self) -> str:
        """Hash only the sections this stage uses, so editing another phase's settings
        does not invalidate this stage's cache."""
        if not self.config_keys:
            return config_hash(self.cfg)
        return config_hash({k: self.cfg.get(k) for k in self.config_keys})

    def is_cached(self) -> bool:
        """True if artifact + metrics exist, the config is unchanged and the gate passed."""
        if not (self.artifact.exists() and self.metrics_path.exists()):
            return False
        try:
            saved = json.loads(self.metrics_path.read_text(encoding="utf-8"))
        except json.JSONDecodeError:
            return False
        return saved.get("cfg") == self.cache_key() and saved.get("passed") is True

    def execute(self, gt=None, force: bool = False) -> bool:
        if not force and self.is_cached():
            if self.log:
                self.log.info("[%s] cached (config unchanged), skipping", self.name)
            return True
        t0 = time.time()
        self.run()
        metrics = self.evaluate(gt)
        ok, reason = self.gate(metrics)
        metrics.update(
            stage=self.name,
            video_id=self.video_id,
            runtime_s=round(time.time() - t0, 3),
            passed=ok,
            reason=reason,
            cfg=self.cache_key(),
            python=platform.python_version(),
        )
        self.metrics_path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
        if self.log:
```

```
            self.log.info("[%s] gate %s: %s", self.name, "PASSED" if ok else "FAILED", reason)
        return ok
```

## 11. requirements.txt [UPDATED]

Adds faster-whisper, jiwer, srt.

```
# Phase 0/1 dependencies. After your first successful install run
# `pip freeze > requirements.lock.txt` and keep that file for exact reproducibility.
# System tools (not pip): FFmpeg + ffprobe must be on PATH.
PyYAML>=6.0,<7
numpy>=1.26,<3
opencv-python>=4.8,<5
yt-dlp>=2024.1.1
pytest>=8.0,<10
# Phase 2 (speech recognition)
faster-whisper>=1.0         # default backend; or: pip install openai-whisper (backend: openai-whisper)
jiwer>=3.0                  # WER / CER
srt>=3.5                    # SRT read/write
```

## 12. env_check.py [UPDATED]

Checks faster_whisper, jiwer and srt as well.

```python
"""Phase 0 environment check: imports, versions, CUDA, ffmpeg -> results/environment_report.json."""
from __future__ import annotations

import importlib
import json
import platform
import shutil
import subprocess
import sys
from datetime import datetime
from pathlib import Path

REQUIRED = ["yaml", "numpy", "cv2", "pytest"]                # needed from Phase 1
OPTIONAL = ["torch", "faster_whisper", "whisper", "jiwer", "srt", "yt_dlp", "scenedetect",
            "insightface", "open_clip"]


def version_of(name: str):
    try:
        mod = importlib.import_module(name)
        return getattr(mod, "__version__", "ok")
    except Exception as exc:                                 # noqa: BLE001
        return f"MISSING ({type(exc).__name__})"


def tool_version(tool: str):
    if shutil.which(tool) is None:
        return None
    out = subprocess.run([tool, "-version"], capture_output=True, text=True)
    return out.stdout.splitlines()[0] if out.returncode == 0 and out.stdout else None


def main() -> int:
    report = {"python": platform.python_version(), "platform": platform.platform()}
    failed = []
    for name in REQUIRED:
        report[name] = version_of(name)
        if str(report[name]).startswith("MISSING"):
            failed.append(name)
    for name in OPTIONAL:
        report[name] = version_of(name)
    for tool in ("ffmpeg", "ffprobe"):
        report[tool] = tool_version(tool)
        if report[tool] is None:
            failed.append(tool)
    try:
        import torch
        report["cuda"] = bool(torch.cuda.is_available())
    except Exception:                                        # noqa: BLE001
        report["cuda"] = False
    report["timestamp"] = datetime.now().isoformat(timespec="seconds")

    out = Path("results") / "environment_report.json"
    out.parent.mkdir(exist_ok=True)
    out.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
    if failed:
        print("FAILED:", ", ".join(failed), file=sys.stderr)
        return 1
    print("Environment OK")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

# Test results

`python -m pytest -q tests`: 38 passed (15 Phase 1 + 23 Phase 2).

| New test | What it proves |
|---|---|
| test_long_segment_is_split_to_balloon_size | no line over 12 words / 4 s, no word lost or reordered |
| test_split_prefers_comma_and_never_makes_one_word_pieces | cuts at clause breaks, no one-word balloons |
| test_split_at_sentence_end_and_long_pause | sentence ends and long pauses start new segments |
| test_short_fragments_are_merged / ..._far_from_neighbours_is_kept | merge only with a close neighbour |
| test_hallucinations_dropped_and_reported | known phrases and no-speech junk dropped, real sentences kept |
| test_overlapping_segments_become_non_overlapping | sorted, non-overlapping output |
| test_segments_clamped_to_audio_duration | nothing runs past the end of the audio |
| test_missing_word_timestamps_are_estimated | fallback when a backend gives no word times |
| test_segment_ids_sequential_from_one / test_validate_timeline_detects_problems | ids and timeline checks |
| test_wer_cer_known_values / test_start_time_mae_pairs_by_text / test_hallucination_rate_... | metric code is correct on hand-computed cases |
| test_srt_round_trips | SRT parses back to the same text and times |
| TestStage (6 tests) | artifacts written; gate passes with good ground truth and fails with bad; cache and config change; needs Phase 1; no-audio video |